THE PROBLEM: VISION & SLAM COMPLEXITY

The Operational Challenge:

- Quadcopters operating in GPS-denied, cluttered surroundings (structural inspections, disaster search & rescue) must navigate autonomously without human pilots.
- Environments contain unmapped obstacles, narrow passages, sharp corners, and dead-ends.
- Traditional solutions employ **Computer Vision (Visual SLAM, optical flow, deep CNNs)** or multi-beam spinning LiDARs.

Limitations of Conventional Vision & SLAM:

- Heavy computational and memory footprint requiring heavy GPUs/SBCs.
- Excessive electrical power consumption severely degrading flight endurance.
- Complex SLAM map generation fails in featureless or reflective corridors.

THE SOLUTION: ADAPTIVE NEURAL CONTROL

The Bio-Inspired Neural Paradigm:

- Inspired by insect reactive navigation, this research replaces heavy vision pipelines with an ultra-lightweight **two-neuron recurrent neural network (RNN)**.
- Operates on **only two lightweight, narrow-beam (2°) LiDAR sensors** mounted at the front of the drone frame.
- **Short-Term Memory (STM):** Retains turning direction even after obstacles leave the narrow LiDAR field of view.
- **Synaptic Plasticity:** Dynamically modulates recurrent weights online to escape corners and deadlocks without global mapping.

Simulation Platform Objective:

- Build an interactive 3D simulation with full 6-DOF physics, an Arduino C++ flight control interface, a Human-in-the-Loop decision gate, and automated telemetry evaluation.

METHODOLOGY & CLOSED-LOOP FLOW

Published Research Citation:
Arne Devos (KU Leuven), Emad Ebeid (SDU UAS Center), Poramate Manoonpong (Embodied AI & Neurorobotics Lab, SDU), *IEEE 21st Euromicro DSD 2018*, pp. 707–710.

Closed-Loop Flight Architecture:

1. **Physical Sensing:** Two Garmin LIDAR-Lite v3 sensors measure obstacle range at 50 Hz.
2. **Sensory Normalization:** Distance mapped non-linearly to discrete [-1.0, +1.0] interval.
3. **Recurrent Controller:** 2-neuron discrete-time neural network computes activations O1, O2.
4. **Synaptic Plasticity:** Correlation learning updates weights bi, qi in real-time.
5. **Steering Generation:** Yaw velocity commanded to flight controller via MAVLink / serial link.
6. **Actuation:** Pixhawk / Arduino commands 4-channel ESC PWM motor thrusts.

PHYSICAL HARDWARE SPECIFICATIONS

Hardware & Software Stack from Base Paper:

Component	Paper Specification	Role / Purpose
Frame	Tarot 650 Sport (Carbon)	Rigid quadcopter frame
Propulsion	370 KV + 14" Props + 40A ESC	High-efficiency thrust
Battery	6S 22.2V 8000 mAh LiPo	25-minute flight endurance
Autopilot	PIXHAWK 2	Attitude IMU & PID loop
Companion SBC	Raspberry Pi Zero	Executes Python RNN loop
Sensors	2x Garmin LIDAR-Lite v3	I2C narrow 2° distance beam
Protocol	MAVLink / MAVProxy / DroneKit	UART serial command bridge

EQUATION (1) & GEOMETRIC FORMULATION

Equation (1) Derivation (Devos et al. 2018):

To guarantee that a quadcopter turning at distance Lx does not collide with obstacles using its rotor tips, the minimum detectable obstacle width *W_obj* is governed by:

$$W_{obj} \geq 2 \cdot \tan(\Theta / 2) \cdot [\, Lx + (Ld / 2) \,]$$

Geometric Parameter Definitions:

- Θ **(LiDAR Separation Angle):** 40° total angular separation (±20° relative to forward axis).
- **Lx (Safe Turning Distance):** 0.15 m (15 cm) distance at which evasion turn begins.
- **Ld (Quadcopter Frame Width):** 0.34 m (34 cm) motor-to-motor arm span.
- **Lm (Rotor Clearance Margin):** 0.06 m (6 cm) safety buffer for propeller tips.
- **Ls (Maximum LiDAR Range):** 0.50 m (50 cm) active sensing horizon.

ANALYTICAL DERIVATION & SAFETY GUARANTEES

Analytical Evaluation & Safety Bound:

Substituting the base paper experimental parameters into Equation (1):

$\Theta / 2 = 20° \Longrightarrow \tan(20°) \approx 0.36397$

$Lx + (Ld / 2) = 0.15\ m + 0.17\ m = 0.32\ m$

$$W_{obj} \geq 2 \cdot 0.36397 \cdot 0.32\ m = 0.2329\ m \approx 24\ cm$$

Engineering Implications:

- Any obstacle with cross-section ≥ **24 cm** will reliably intercept at least one LiDAR beam before the drone breaches the 15 cm critical threshold.
- Obstacles narrower than 24 cm could pass between the two beams if not angled; therefore, all obstacles in our scenarios have physical widths ≥ 24 cm.
- **Minimum Clearance Boundary:** Lm + (Ld / 2) = 0.06 + 0.17 = **0.23 m**. Any distance below 0.23 m constitutes a geometric rotor collision.

NORMALIZATION EQUATION & TRANSFER FUNCTION

Raw LiDAR to Neural Input Transformation:

The raw distance measurements *d_L, d_R* from the dual front LiDAR sensors cannot be directly fed into neural activation functions. They must be normalized into continuous bipolar signals:

$$S_i = \text{clip}\left(\left[2 \cdot \frac{(d_{max} - d_i)}{(d_{max} - d_{min})} \right] - 1.0, -1.0, 1.0 \right)$$

Where $d_{max} = 0.50\ m$ (sensing horizon Ls) and $d_{min} = 0.15\ m$ (critical turning distance Lx).

State Boundary Interpretations:

- **S_i = -1.0:** Free airspace. Obstacle distance $d_i \geq 0.50\ m$. Zero reflex excitation.
- **S_i = 0.0:** Transition boundary. Obstacle at $d_i = 0.325\ m$. Reflex activates.
- **S_i = +1.0:** Critical proximity. Obstacle at $d_i \leq 0.15\ m$. Maximum turning drive.

DUAL SENSOR ARRANGEMENT & 3D EXPANSION

Dual LiDAR Beam Geometry (Theta = 40°):

- **Left LiDAR (S_L):** Oriented at +20° azimuth relative to drone heading vector.
- **Right LiDAR (S_R):** Oriented at -20° azimuth relative to drone heading vector.
- **Viewing Angle:** 2° narrow laser cone minimizes ground and ceiling multipath reflections.

Multi-Directional Range Array in Simulation:

While the base paper relies strictly on dual front LiDARs for 2D room navigation, this project enhances the spatial perception suite for full 3D research:

- **Planar 8-Beam Array:** Forward, Left, Right, Backward, Forward-Left, Forward-Right, Rear-Left, Rear-Right.
- **Vertical 2-Beam Array:** Upward ceiling laser + Downward ground altimeter laser.
- Feeds omnidirectional clearance telemetry directly to Human-in-the-Loop Decision Gate.

RECURRENT STATE UPDATE EQUATIONS

Discrete-Time Non-Spiking Neural Dynamics:

The core controller (Pedersen & Manoonpong 2018) consists of two recurrently coupled neurons with hyperbolic tangent nonlinearities:

$$O_1(t) = \tanh(b_1(t)\cdot O_1(t-1) + c(t)\cdot O_2(t-1) + I_g\cdot S_L(t))$$
$$O_2(t) = \tanh(b_2(t)\cdot O_2(t-1) + c(t)\cdot O_1(t-1) + I_g\cdot S_R(t))$$

Mathematical Terms & Roles:

- **$O_1(t), O_2(t) \in$ (-1.0, 1.0):** Left and Right steering neuron activation states.
- **$O_1(t-1), O_2(t-1)$:** Prior states acting as **Short-Term Memory (STM)**.
- **$b_1, b_2 \in$ [0.1, 8.0]:** Self-excitatory recurrent synapses (baseline = 2.40). Sustains turning commands after sensor signal drops.
- **$c(t) = -0.5 \cdot [q_1(t) + q_2(t)]$:** Mutual cross-inhibitory synaptic coupling (baseline = -3.50). Prevents contradictory turning.
- **I_g = 4.7:** Sensory input gain amplifying normalized LiDAR signals.

NEURAL COMPETITION & EFFICIENCY ADVANTAGES

Winner-Take-All Directional Competition:

- When the quadcopter detects an obstacle on the left, *S_L* increases. Neuron 1 excites rapidly while cross-inhibiting Neuron 2 via *c(t)*.
- This causes **O_1** $\rightarrow$ **+1.0** and **O_2** $\rightarrow$ **-1.0**, creating an unequivocal steering command to the right.
- In head-on symmetry (both sensors stimulated equally), small numerical asymmetries or historical turning bias break symmetry instantly, avoiding stagnation.

Comparison with Deep Neural Networks:

- **Parameters:** Only 4 adaptable synaptic weights vs millions in CNNs.
- **Compute:** Sub-microsecond execution time on low-cost Arduino / RPi Zero.
- **No Training Needed:** Operates online with closed-loop stability.

THE DEADLOCK PROBLEM & PLASTICITY EQUATIONS

The Corner & Deadlock Failure Mode:

- When an autonomous drone enters a 90° corner or U-turn deadlock, it begins turning. Because LiDARs have narrow 2° beams, the wall leaves the sensor field of view before the turn finishes.
- Without plasticity, the drone stops turning too early, accelerates forward, and crashes into the adjacent wall.

Correlation-Based Synaptic Plasticity Formulation:

Synaptic weights *b_i* (self-feedback) and *q_i* (inhibition) adapt dynamically using correlation learning (Pedersen & Manoonpong 2018):

$$v_i(t) = clip([O_i(t) + 1] / 2, 0.0, 1.0)$$

$$reflex_i(t) = 1.0 \text{ if } S_i(t) > -0.5 \text{ else } 0.0$$

$$\Delta b_i = \kappa \cdot \Delta t \cdot [\mu_b \cdot v_i(t-1) \cdot v_i(t) \cdot reflex_i + \gamma_b \cdot (k - v_i) \cdot b_i^2]$$

$$\Delta q_i = \kappa \cdot \Delta t \cdot [\mu_q \cdot v_i(t-1) \cdot v_i(t) \cdot reflex_i + \gamma_q \cdot (k - v_i) \cdot q_i^2]$$

HYPERPARAMETERS & DUAL-PHASE ADAPTATION

Plasticity Hyperparameters (Base Paper):

- κ **(Scaling rate):** 27.0 — Governs learning adaptation timescale.
- μ_b, μ_q **(Learning rates):** $\mu_b = 0.0065$, $\mu_q = 0.015$.
- γ_b, γ_q **(Decay rates):** $\gamma_b = 0.0003$, $\gamma_q = 0.0003$.
- **k (Target baseline):** -0.01 — Homeostatic resting point.

Dual Mechanism Explanation:

1. **Hebbian-like Potentiation (1st term):** When obstacle reflex is active, consecutive neuron firing strengthens *b_i* and *q_i*, lengthening short-term memory persistence.
2. **Homeostatic Synaptic Scaling (2nd term):** When the drone escapes into open air, weights decay back to baseline resting states, preventing over-turning.

Result: 90% to 100% autonomous navigation success in tight deadlocks.

YAW RATE COMMAND GENERATION

Differential Yaw Steering Command:
The net steering angle command is derived from the difference between the two recurrent neuron outputs:

$$steering(t) = 0.5 \cdot [\, O_1(t) - O_2(t) \,]$$
$$\psi_cmd(t) = -k_turn \cdot steering(t)$$

Where *k_turn = 1.35 rad/s* (yaw rate gain).

Directional Evasion Logic:

- **Left Obstacle (S_L > S_R):** O_1 > O_2 ==> steering > 0 ==> **ψ_cmd < 0 (Turn Right)**.
- **Right Obstacle (S_R > S_L):** O_2 > O_1 ==> steering < 0 ==> **ψ_cmd > 0 (Turn Left)**.
- **Symmetric Clear Flight:** O_1 ≈ O_2 ==> steering ≈ 0 ==> Straight cruise.

VELOCITY REGULATION & FLIGHT DYNAMICS

Adaptive Forward Cruise Velocity Throttling:
To guarantee that the quadcopter can safely negotiate turns within its turning radius, forward velocity *V_forward* throttles dynamically:

V_forward = 0.00 m/s if S_L > 0.55 and S_R > 0.55 (Deadlock)
V_forward = 0.025 m/s if max(S_L, S_R) > 0.25 (Very Close)
V_forward = 0.055 m/s if max(S_L, S_R) > -0.15 (Approaching)
V_forward = 0.10 m/s otherwise (Nominal Cruise Speed)

Pitch/Roll Feedforward Dynamics:

- 6-DOF physics engine couples forward speed to pitch tilt angle $\theta \approx atan(V_f / g)$ and yaw rate to roll banked coordinated turns.
- Ensures realistic flight dynamics matching Tarot 650 physical inertia.

ARDUINO C++ FIRMWARE FEATURES

Arduino Firmware (arduino_flight_controller.ino):
The project includes complete, production-ready C++ firmware for Arduino Mega 2560 / Uno / ESP32 platforms:

- **Pin Mappings:** 4-channel ESC PWM outputs (D5, D6, D9, D10) and ultrasonic/LiDAR trigger & echo pins.
- **Dual Range Sampling:** 50 Hz non-blocking distance acquisition matching Devos et al. timing.
- **Cascaded PID Control:** High-rate inner angular rate PID + outer attitude angle PID for Roll, Pitch, and Yaw.
- **ESC Motor Mixing:** X-quadcopter motor matrix translating throttle, roll, pitch, and yaw commands into individual motor RPMs.
- **Failsafe Protection:** Immediate motor cut-off on loss of serial heartbeat or emergency stop signal.

HITL/SITL SERIAL PROTOCOL & DUAL MODES

SITL & HITL Serial Bridge (serial_bridge.py):

- **Bidirectional ASCII Telemetry Protocol:** Transmits distance, orientation, and armed status strings (*$TELEM, dL, dR, roll, pitch, yaw*).
- **Auto-Detection:** Scans available COM ports for CH340, FTDI, or ATmega32u4 USB controllers.
- **Hardware-in-the-Loop (HITL):** When connected, the Arduino receives Python commands and actuates real motor PWMs.
- **Software-in-the-Loop (SITL):** If no physical hardware is plugged in, the system seamlessly simulates Arduino telemetry without errors.

Physical Airframe Baseline: Tarot 650 Sport, 370 KV motors, 14" stiff carbon fiber propellers, 40A ESCs, 6S 22.2V 8000 mAh LiPo battery (25 min flight).

7-STAGE DECISION PIPELINE

Interactive Decision Philosophy:
Unlike black-box simulators that avoid automatically without user input, this simulation implements an explicit **Human-in-the-Loop Decision Gate**:

1. **Cruise:** Quadcopter advances forward at 0.10 m/s in steady hover.
2. **Detection:** Sensor detects obstacle within 0.50 m threshold.
3. **Simulation Pause:** The physics clock halts progression immediately.
4. **Safe Hover Hold:** Drone stabilizes in safe hover waiting for operator.
5. **Safety Audit:** Evaluates line-of-sight clearance across all 10 candidate directions against Equation (1) geometric clearance bounds.
6. **Operator Decision:** User selects evasion direction via UI or hotkeys.
7. **Execution & Logging:** Maneuver executes, trajectory updates in 3D, and trial metrics are permanently logged.

SAFETY AUDIT & RESEARCH OVERRIDE

Direction Classification & Safety Criteria:

Status	Clearance	System Behavior
RECOMMENDED	> 0.50 m	Optimal base-paper neural evasion direction
SAFE	> 0.50 m	Clear open airspace; execution permitted
CAUTION	0.23m - 0.50m	Restricted clearance; warning modal shown
BLOCKED	< 0.23 m	Rotor collision hazard; blocked without override

Research Override Mode: A dedicated toggle switch permits testing unsafe choices deliberately to inspect collision physics and damage telemetry.

3D VIEWPORT & VISUALIZATION

Web Dashboard Features (http://localhost:8080):

- **High-Fidelity 3D Viewport:** Built with Three.js WebGL. Renders carbon-fiber Tarot 650 quadcopter with spinning rotors, dynamic shadows, and 3D arena wireframe.
- **LiDAR Laser Visualizer:** Renders active dual front LiDAR beams (±20°) and omnidirectional raycasters with distance color-coding (Green/Yellow/Red).
- **Flight Trajectory Ribbon:** 3D continuous flight path visualizer tracking evasive maneuvers in real time.
- **4 Dynamic Camera Modes:**

1. *Chase Cam:* Smooth follow-camera behind the quadcopter.
2. *Orbit Cam:* Free 360° mouse orbital inspection.
3. *Tactical Top:* 2D top-down orthogonal spatial overview.
4. *FPV Nose:* First-person pilot perspective from drone nose.

CONTROLS, HOTKEYS & TELEMETRY

Interactive Controls & Telemetry HUD:

- **10 Flight Direction Controls:** Forward, Backward, Left, Right, Up, Down, Front-Left, Front-Right, Rear-Left, Rear-Right.
- **Keyboard Hotkeys:** Full desktop control (W/A/S/D planar, Q/E/Z/C diagonals, R/F vertical, Space start/pause, Enter confirm, Esc stop).
- **Dual Pilot Modes:** Instant toggle between Autonomous Devos (2018) neural mode and Manual Pilot stick flight.
- **Live Telemetry HUD:** Altitude, Airspeed, Yaw Heading, Clearance meters, and RNN neuron activations (O1, O2, b1, b2, q1, q2).
- **Simulation Clock:** 0.5x, 1.0x, 2.0x, 5.0x simulation speed controls.
- **WebSocket Bridge:** Low-latency 30 Hz bi-directional JSON stream.

ACTIVE 13-SCENARIO SUITE (CONSTRAINED PASSAGE, MOVING & ENV 1 REMOVED PER USER INSTRUCTION)

Scenario ID	Scenario Name	Threat Geometry & Avoidance Objective
front_obstacle	Front Obstacle (Direct Ahead)	Cylindrical pillar directly in forward flight path. Test left/right turn.
right_obstacle	Right Flank Obstacle	Wall barrier on right side. Test leftward evasion or straight cruise.
left_obstacle	Left Flank Obstacle	Barrier on left flank. Test rightward evasion.
rear_obstacle	Rear Obstacle (Reverse Threat)	Obstacle behind quadcopter; tests rear raycast during reverse flight.
above_obstacle	Above Obstacle (Overhead Bridge)	Low ceiling / overhead pipe. Tests vertical descend or lateral avoidance.
below_obstacle	Below Obstacle (Raised Platform)	Elevated ground structure. Tests downward sensor and vertical climb.
both_sides	Both Sides Flanked (Lateral Pinch)	Obstacles on both sides. Evaluates forward narrow escape or reverse.
front_left	Front-Left Diagonal Obstacle	Threat at +45° quadrant. Tests front-right diagonal escape.
front_right	Front-Right Diagonal Obstacle	Threat at -45° quadrant. Tests front-left diagonal escape.
rear_diagonal	Rear Diagonal Obstacles	Pillars at rear-left and rear-right quadrants. Forward escape.
multiple_obstacles	Multiple Clustered Obstacles	Dense field of 7 pillars and boxes evaluating multi-sensor perception.
paper_env2_maze	Base Paper Env 2: Maze & Deadlocks	Devos et al. Fig 5(b) reconstruction with sharp corners and deadlocks.
clear_flight	Clear Flight (No Obstacles)	Open arena baseline verifying flight dynamics, hover, and trims.

EXHAUSTIVE SIDE-BY-SIDE METHODOLOGICAL MAPPING

Architectural Dimension	Base Research Paper (Devos et al. 2018)	Simulation Platform Implementation
Sensing Array	Dual front LiDARs at ±20° in 2D horizontal plane	Exact dual front LiDARs (Θ = 40°, Ls = 50cm) plus 8-direction radial and Up/Down 3D sensor suite
Avoidance Trigger	Purely autonomous reactive closed-loop control	Interactive Human-in-the-Loop decision gate: pauses on detection, holds hover, audits safety, prompts user
Maneuver Options	Continuous yaw steering only (Turn Left / Right)	10 degrees of freedom (Forward, Reverse, Left, Right, Climb, Descend, 4 Diagonals, Hover, E-Stop)
Simulation Engine	V-REP robotics simulator with C++ ROS interface	Full 6-DOF Quadcopter Physics + Three.js 3D WebGL Dashboard with 30 Hz async WebSocket stream
Embedded Firmware	Pixhawk 2 autopilot + Raspberry Pi Zero companion SBC	Python neural backend + complete Arduino C++ firmware (arduino_flight_controller.ino) with SITL/HITL
Environments	2 static test maps (Room exploration & Maze)	13 scenarios covering all spatial threat angles, clusters, and reconstruction of Paper Env 2
Safety Evaluation	Empirical flight trial observation	Automated mathematical clearance audit + Research Override Switch to test collisions deliberately
Logging & Analytics	Offline video review and path extraction	Real-time automated logging with one-click export to CSV and JSON formats

DATA COLLECTION & LOGGING SCHEMA

Experiment Logging Schema:

Every flight trial automatically records a comprehensive experiment record:

- **Trial Identification:** Unique timestamped ID and active scenario ID.
- **Flight Coordinates:** Initial drone position and final coordinates.
- **Sensory Snapshot:** Exact distance readings on all sensors at moment of detection trigger.
- **Decision Telemetry:** Available safe directions, user-selected direction, executed maneuver, and override flag.
- **Outcome & Clearance:** Success vs Collision status, collision object ID, minimum observed clearance, and avoidance duration.
- **Operator Response Time:** Latency (s) from safe hover alert to directional input.

QUANTITATIVE METRICS & BENCHMARKS

Quantitative Evaluation Metrics (Logged in CSV/JSON):

- **Success Rate (%):** Proportion of avoidance trials completed without rotor breach.
- **Collision Rate (%):** Proportion of trials resulting in collision (primarily during research override testing).
- **Mean Response Time (s):** Average latency taken by human/controller.
- **Minimum Clearance (m):** Minimum distance between drone radius and obstacles throughout the entire evasion trajectory.

Base Paper Benchmark Comparison:

- *Low Obstacle Density:* 100% collision-free navigation.
- *High Obstacle Density:* 90% collision-free navigation.
- *Deadlocks & Corners:* 90% successful escape without human intervention.
- **Instant Data Export: Single-click export to CSV and JSON formats.**

PROJECT DIRECTORY & MODULES

Modular Project Directory Structure:

```
arduino_quadcopter_simulation/
■■■ main.py (aiohttp async server, 30 Hz loop)
■■■ METHODOLOGY.md (Formal mathematical reference)
■■■ requirements.txt (aiohttp, numpy, scipy, pytest)
■■■ simulation/
■ ■■■ drone_physics.py (6-DOF quadcopter dynamics)
■ ■■■ scenarios.py (13 active test configurations)
■ ■■■ experiment_logger.py (CSV/JSON trial logging)
■■■ avoidance/
■ ■■■ neural_controller.py (Devos 2-neuron RNN + plasticity)
■ ■■■ decision_engine.py (Direction safety audit & HITL)
■■■ sensors/ lidar_sensor.py (Dual LiDAR + 3D raycasting)
■■■ dashboard/ (Three.js WebGL, CSS glassmorphism, app.js)
■■■ arduino/ (arduino_flight_controller.ino, serial_bridge.py)
■■■ scenarios/ (13 exported JSON scenario definitions)
■■■ tests/ (Comprehensive automated pytest test suite)
```

AUTOMATED UNIT TESTING & VERIFICATION

Automated Unit Testing Suite (pytest):

- **test_avoidance.py (7 tests):** Validates distance normalization boundaries (0.5m → -1, 0.15m → +1), tanh neuron bounds (-1, +1), steering signs (Left obstacle → Right turn), synaptic plasticity updates, and HITL state machine.
- **test_physics.py (4 tests):** Validates 2D circle/box raycasting math, 6-DOF translation dynamics, and collision boundary detection.
- **test_scenarios.py (3 tests):** Validates scenario library integrity, CSV/JSON trial export, and steps through all 13 scenarios with zero collisions.
- **Execution: python -m pytest tests/ — 14/14 tests pass.**

Deployment & Launch:

- **Server online at: http://localhost:8080/**
- Command: python main.py (persistent background task).

SUMMARY OF CONTRIBUTIONS

Key Scientific & Engineering Contributions:

1. **Faithful Research Reproduction:** Rigorously implemented all equations, parameters, and algorithms from Devos et al. (2018) IEEE DSD and Pedersen & Manoonpong (2018).
2. **Ultra-Low Compute Footprint:** Demonstrated that complex corner navigation and deadlock escape are achievable using just 2 non-spiking tanh neurons and 4 synaptic weights.
3. **Dual LiDAR Geometry:** Validated Equation (1) bounding minimum detectable obstacle width at $W_{obj} \geq 24$ cm for 15 cm turning thresholds.
4. **Interactive Human-in-the-Loop Workflow:** Replaced opaque autonomous loops with a transparent decision gate where human operators inspect clearance geometry and choose directions.
5. **Hardware-Ready Arduino Integration:** Delivered complete C++ firmware and serial bridge supporting real-world hardware deployment.

FUTURE WORK & PROJECT ACCESS

Future Roadmap & Research Directions:

- **Physical Flight Validation:** Transfer Arduino firmware to Tarot 650 quadcopter for real-world GPS-denied tunnel inspections.
- **ROS 2 & Gazebo Integration:** Bridge the Python neural controller to ROS 2 Humble micro-XRCE-DDS for swarm coordination.
- **Dynamic Obstacle Tracking:** Extend synaptic plasticity to predict velocities of moving obstacles in non-stationary environments.
- **Multi-Agent Collaborative Evasion:** Distributed neural consensus among quadcopters in shared corridors.

Active Environment Link:

`http://localhost:8080/`

Live 3D simulation with 13 active scenarios, full keyboard control, and telemetry logging.